Pautlen

Pautlen UAM | 2020 - 2021

Authors

> Víctor Perea Riquelme | victor.perea@estudiante.uam.es

> Álvaro Rodríguez Palacios | alvaro.rodriguezp@estudiante.uam.es

> Javier Romera Llave | javier.romeral@estudiante.uam.es

Assignment 3 - Syntactic

Makefile

For help, run:

```
make help
```

```
❯ make help
all                             Compile all.
clean                           Cleans output files
default                         Equivalent to 'make all'
exe_manual                      Execution as (stdin, stdout):(yyin, yyout)
help                            This guide
manual                          Same as exe_manual without recompiling
test_all                        Executes all tests
test_first                      Executes first test
test_second                     Executes second test
test_third                      Executes third test
```

Folder structure

Important: shown tree before `make` and after `make clean`:

```
root
.
├── alfa
│   ├── alfa.c
│   ├── alfa.l
│   └── alfa.y
├── inc
│   ├── alfa.h
│   ├── rules.h
│   └── tokens.h
├── Makefile
├── obj
├── out
├── README.html
├── README.md
├── README.pdf
├── src
├── testfiles
│   ├── entrada_sin_1.txt
│   ├── entrada_sin_2.txt
│   ├── entrada_sin_3.txt
│   ├── salida_sin_1.txt
│   ├── salida_sin_2.txt
│   └── salida_sin_3.txt
├── testresults
└── testverify.sh
```

Important: shown tree after `make`, `make test` and before `make clean`

```
root
.
├── alfa
│   ├── alfa.c
│   ├── alfa.l
│   └── alfa.y
├── inc
│   ├── alfa.h
│   ├── rules.h
│   ├── tokens.h
│   └── y.tab.h
├── Makefile
├── obj
│   ├── alfa.o
│   ├── lex.yy.o
│   └── y.tab.o
├── out
│   └── pruebaSintactico
├── README.html
├── README.md
├── README.pdf
├── src
│   ├── lex.yy.c
│   └── y.tab.c
├── testfiles
│   ├── entrada_sin_1.txt
│   ├── entrada_sin_2.txt
│   ├── entrada_sin_3.txt
│   ├── salida_sin_1.txt
│   ├── salida_sin_2.txt
│   └── salida_sin_3.txt
├── testresults
├── testverify.sh
└── y.output
```

Tests

In order to help either us or whoever, we have provided a Makefile rule - `make test_all` - which will throw the result of all test. Also separately:

`make test_first`

`make test_second`

`make test_third`

This rules execute **pruebaSintactico** with it's correspondant source testfile - located at *testfiles/* - and, helped by **testverify.sh**, a script which process the result of doing `diff -Bb inputfile.txt outputfile.txt`, shows a message with the result of the test. Expected return for diff is '\0'.

testverify.sh

```
#!/bin/bash

# info: checks if file_source is same to file_result
# args:
#     (0): script exec
#     (1): file_source
#     (2): file_result

ON_ERROR="TEST FAILURE."
ON_SUCCESS="TEST SUCCESS."

red=$'\e[1;31m'
grn=$'\e[1;32m'
end=$'\e[0m'

# Main

file_source=$1
file_result=$2

if [ $# -eq 2 ]; then
  printf 'Test Result: '
  DIFF=$(diff -Bb $file_source $file_result)
  if [ "$DIFF" != "" ]; then
    printf "${red}%s${end}\n" "$ON_ERROR"
    printf "Difference:\n"
    diff -Bb $file_source $file_result
  else
    printf "${grn}%s${end}\n" "$ON_SUCCESS"
  fi
else
  printf 'Bad args: %s FILE_SOURCE FILE_RESULT\n' "$0"
fi
```

For manual testing

If manual testing is deserved, you can type `make exe_manual` to compile and next time `make manual`. This will run **pruebaSintactico** with *stdin* and *stdout* as *yyin* and *yyout* respectively.

Alfa.c, Alfa.y and Alfa.l and Common Header

Code is run from **main()** which is in **alfa.c**; *yyin* and *yyout* set up from desired option (if manual standards ones, else specified files), then **yyparse()** is called.

yyparse() - Bison's parser - will "talk" with **yylex()** - Flex's parser -, whose return value will be used by Bison's function in order to build and complete syntactic tree, and then give feedback about how has been the result - exit code and file type and source error -.

Common header file **alfa.h** contains `extern char *errbuff`, a common buffer to manage errors detected in **alfa.l** and handled at **alfa.c**, as well as `*extern int line*`, `*extern int column*` for error's feedback purposes, also `*extern bool morfofailure*` to distinguish if failure comes from **alfa.l** or from **alfa.y**.